



Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

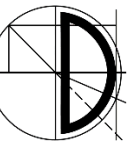


Grundlagen der Programmierung

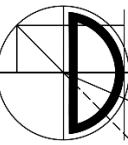
Kapitel 5: Anweisungen und Ablaufsteuerung (Teil 2)

Prof. Dr. Samuel Kounev, Daniel Grillmeyer M.Sc.

- Alle Materialien (inkl. Vorlesungs- und Übungsfolien, Vorlesungs- und Übungsaufzeichnungen, Übungsblätter, Programmieraufgaben) in diesem Kurs werden nur für Ihre persönliche Nutzung bereitgestellt. Das Teilen oder Weitergeben der Inhalte an Dritte ist generell untersagt!
- Vorlesungsmaterialien von Prof. Dr. Detlef Seese wurden als Basis verwendet
- Unterstützung bei der technischen und inhaltlichen Gestaltung des Vorlesungsmaterials leisteten: Martin Sträßer, Daniel Grillmeyer, Jóakim v. Kistowski, Dietmar Ratz, Joachim Melcher, Roland Küstermann, Jana Weiner, Hagen Buchwald, Matthes Elstermann, Oliver Schöll, Niklas Kühl, Tobias Diederich



- **while**-, **do**-, **for**-Schleifen
- Endlosschleifen
- Sprungbefehle **break** und **continue**
- Markierte Anweisungen



Syntax einer `while`-Anweisung

```
while (<Bedingung>)
```

```
    <Anweisung>
```

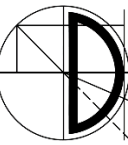
```
while (<Bedingung>) {
```

```
    <Anweisungsfolge>
```

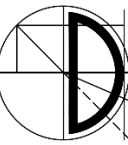
```
}
```

Schleifenkörper
oder Rumpf
(loop body)

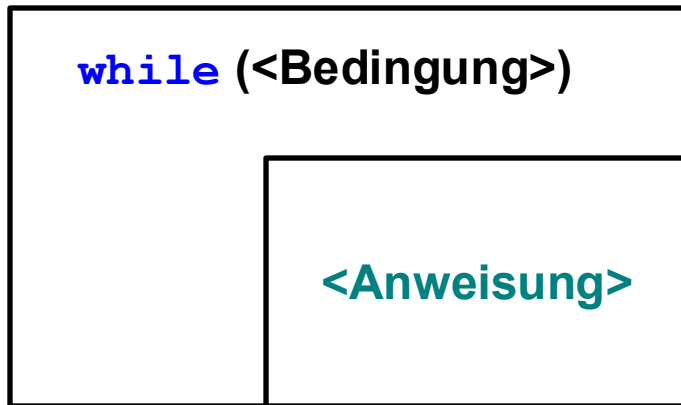
<Bedingung> = Boolescher Ausdruck



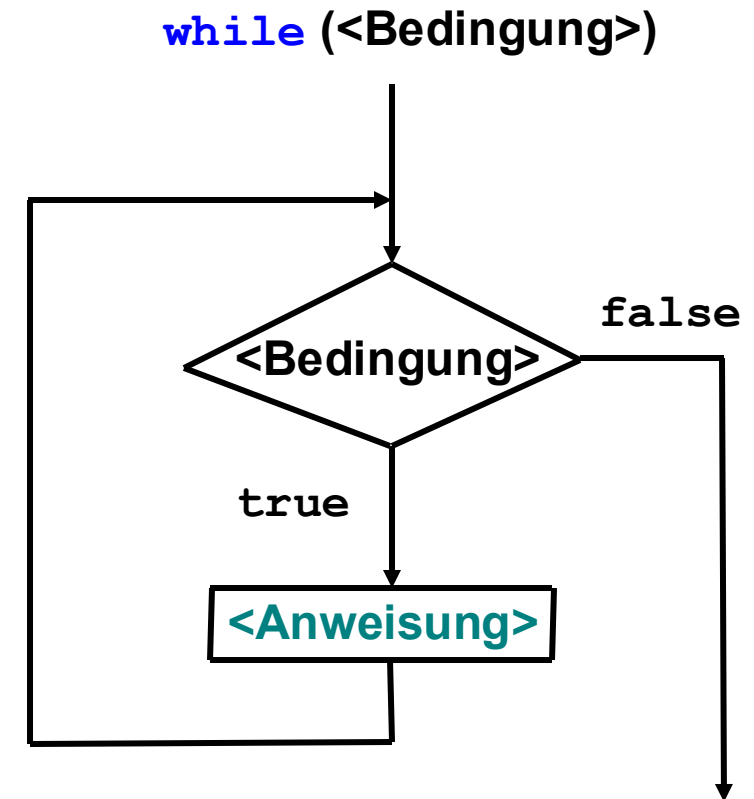
- Semantik:
 - prüfe die Bedingung
 - ist diese wahr, so wird der Körper ausgeführt
 - ist die Bedingung falsch, so wird die Schleife verlassen, d.h. die Anweisungen nach dem Schleifenkörper werden bearbeitet
- Die **while**-Schleife wird auch als abweisende Schleife oder Schleife mit Eingangsprüfung bezeichnet



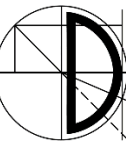
```
while (<Bedingung>)  
    <Anweisung>
```



Struktogramm

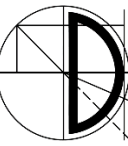


Programmablaufplan (PAP)



```
// Berechnung der Fakultät:  
//  $k! = k * (k-1) * (k-2) * \dots * 3 * 2 * 1$ 
```

```
IO.print("Positive ganze Zahl: ");  
long k = input.nextLong();  
IO.print(k + "! = ");  
long f = 1;  
while (k > 1) {  
    f = f * k;  
    k = k - 1;  
}  
IO.println(f);
```



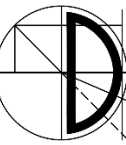
Vorsicht Falle!

```
double x = 15;           // Endlosschleife!  
while (x > 10);          // Rumpf der while-Schleife besteht  
    x = x - 10;          // nur aus leerer Anweisung (wg. ;)
```

```
while (x > 0)             // Nur die erste Anweisung gehört  
    x = Math.sqrt(x);    // zum Rumpf der while-Schleife  
    x = x - 10;
```

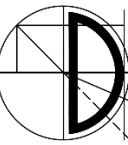
```
while (true)             // Endlosschleife  
    x = 5;
```

```
while (false)            // Compilierungs-Fehler,  
    x = 5;              // da Anweisung unerreichbar
```

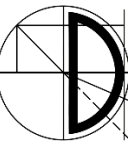



Wie stoppt man eine Endlosschleife?

```
while (true) {  
    // Anweisungen  
    // hier sollte ein break  
    // oder ein return stehen,  
    // return wird später behandelt  
}
```



```
int x = 1;
while (true) {
    IO.println(x);
    if (x == 10) break;
    // unterbricht die aktuelle Schleife
    x = x + 1;
}
```



Syntax einer `do-while`-Anweisung

`do`

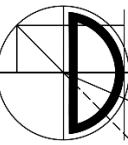
`<Anweisung> oder <Anweisungsblock>`

`while (<Bedingung>);`

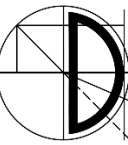
Schleifenkörper
oder Rumpf
(loop body)

`<Bedingung>` = Boolescher Ausdruck

Die do-while-Schleife ist eine Schleife mit Ausgangsprüfung!



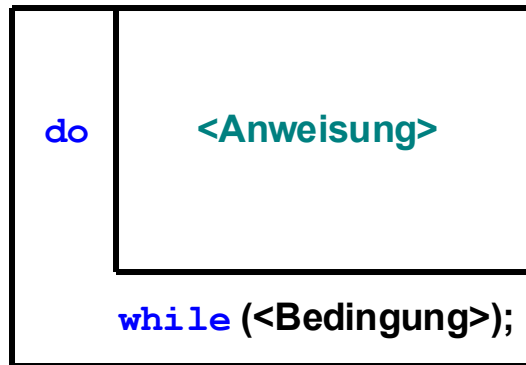
- Semantik:
 - Abarbeitung des Schleifenkörpers
 - Prüfung der Bedingung
 - Ist die Bedingung wahr, wird wieder der Schleifenkörper abgearbeitet
 - ist die Bedingung falsch, so wird die Schleife verlassen
- Daraus folgt: Auch wenn die Bedingung von Anfang an falsch ist, wird der Schleifenkörper in jedem Fall mindestens einmal ausgeführt



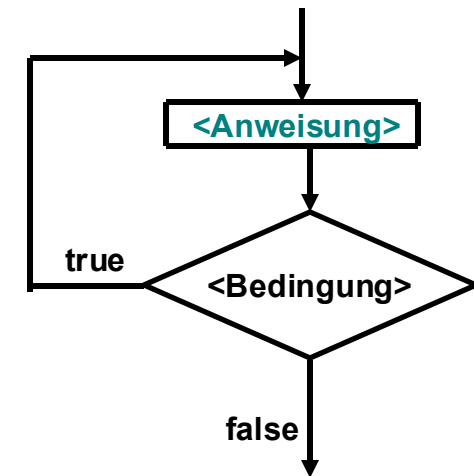
do

<Anweisung> oder <Anweisungsblock>

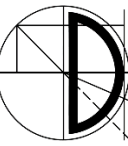
while (<Bedingung>);



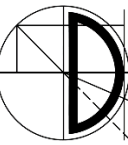
Struktogramm



Programmablaufplan (PAP)



```
int i = 1;  
do {  
    IO.println(i);  
    i = i + 1;  
} while (i < 6);
```



Syntax einer **for**-Anweisung

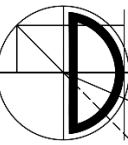
```
for (<Initialisierungsliste>; <Bedingung>; <Updateliste>)  
    <Anweisung> oder <Anweisungsblock>
```

Schleifenkörper
oder Rumpf
(loop body)

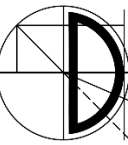
<Initialisierungsliste> = Komma-Liste von Ausdrücken bzw. lokalen Variablendeklarationen mit Initialisierung

<Bedingung> = Boolescher Ausdruck

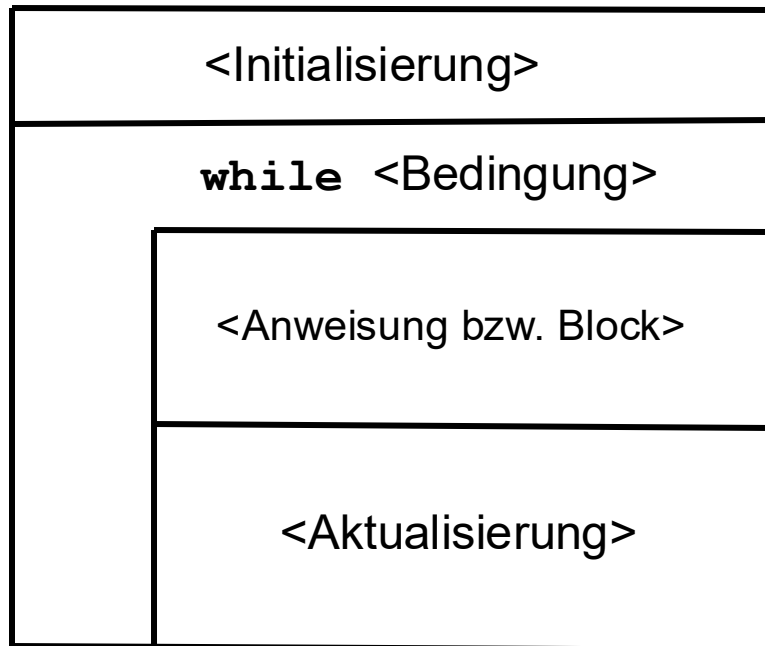
<Update-Liste> = Komma-Liste von Ausdrücken



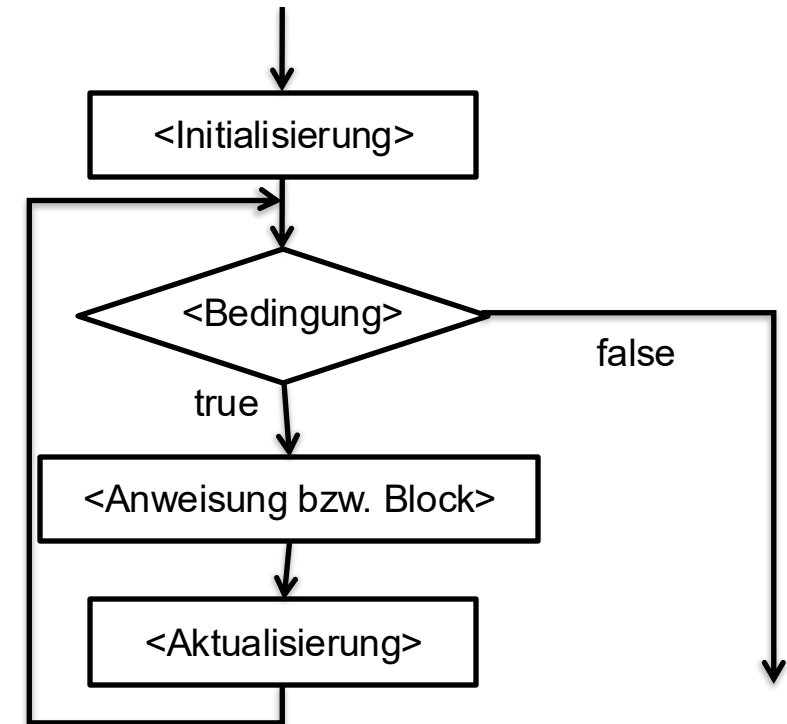
- Semantik:
 - Führe die Initialisierungsliste aus
 - Werte die Bedingung aus
 - Wenn die Bedingung wahr ist
 - Führe den Schleifenkörper aus
 - Führe die Update-Liste aus
 - Prüfe Bedingung erneut
 - Wenn die Bedingung falsch ist, wird die Schleife verlassen



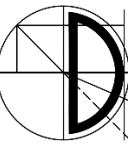
for (<Initialisierung>_{opt}; <Bedingung>_{opt}; <Aktualisierung>_{opt})
 <Anweisung bzw. Block>



Struktogramm



Programmablaufplan (PAP)



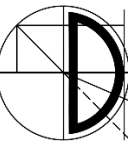
```
// Einmaleins
for (int i = 1; i <= 10; i++) {
    for (int j = 1; j <= 10; j++) {
        IO.println(i + "*" + j + "=" + i*j);
    }
}
```

```
// Endlosschleife  
for (;;) {  
    IO.println("Endlosschleife!")  
}
```

```
// unzulässige Schleifen  
for (int p=0, long q=7; p < q; p++, q--) {  
    // Anweisungen  
}  
int u;  
for (int u=0; u < 10; u++) {  
    // Anweisungen  
}
```

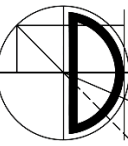
Unzulässige 2.
Typangabe

u ist bereits
vorher deklariert

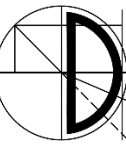


```
// while-Schleife
int i = 1;
while (i < 11) {
    IO.println(i);
    i++;
}
```

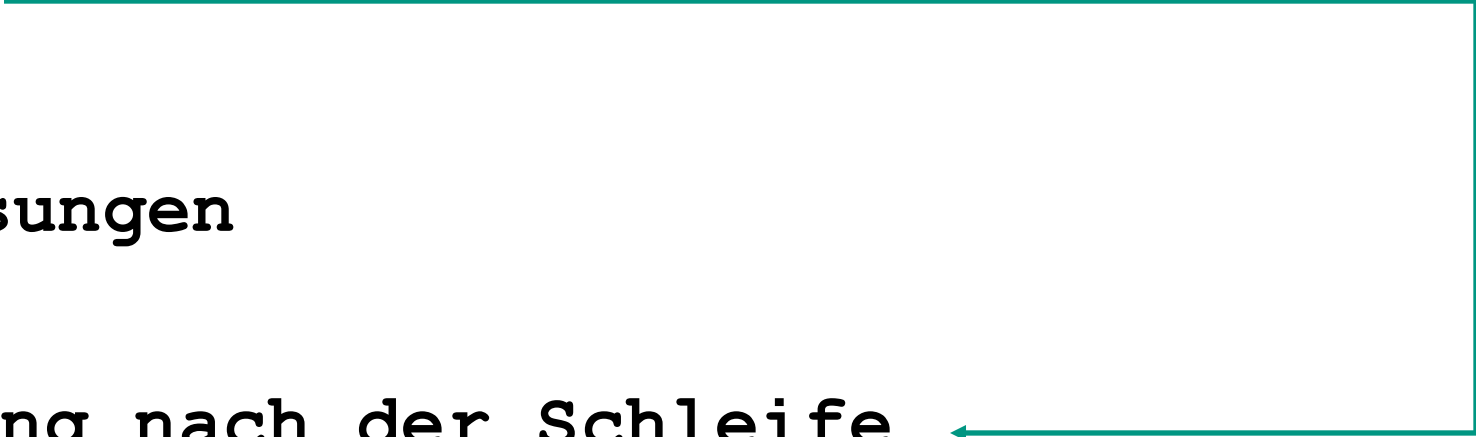
```
// for-Schleife
for (int i = 1; i < 11; i++) {
    IO.println(i);
}
```

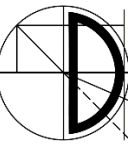


- Beendet eine Schleife oder `switch`-Anweisung
 - Kann innerhalb von `for`-, `while`-, `do-while`- oder `switch`-Anweisungen auftreten
 - `break` beendet die innerste Schleife oder `switch`-Anweisung
 - Um eine äußere Schleife oder `switch`-Anweisung abubrechen kann in Java eine Marke (`label`) verwendet werden

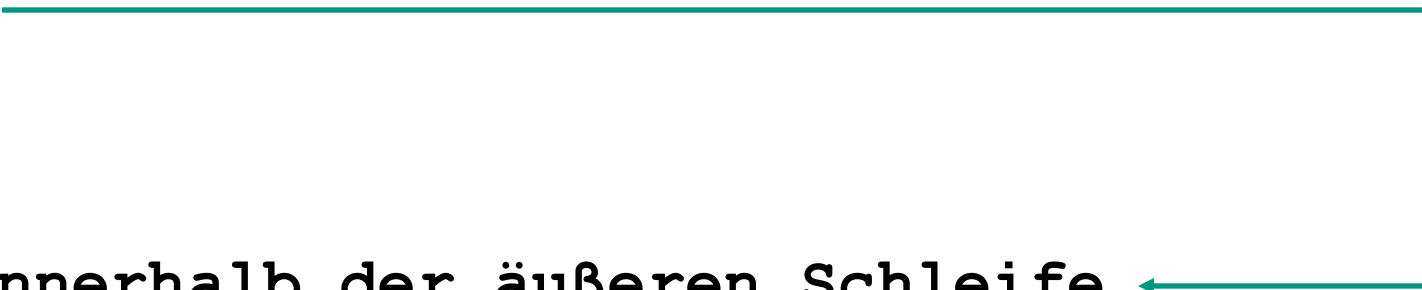


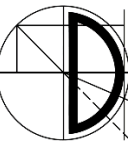
```
while ( Bedingung ) {  
    // Anweisungen  
    if (...) {  
        break;  
    }  
    // Anweisungen  
}  
// 1. Anweisung nach der Schleife
```





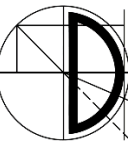
```
while ( Bedingung ) {  
    // Anweisungen  
    while ( Bedingung ) {  
        if (...) {  
            break;  
        }  
    }  
    // Anweisungen innerhalb der äußeren Schleife  
}  
// 1. Anweisung nach äußerer Schleife
```



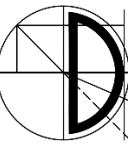


outerLoop:

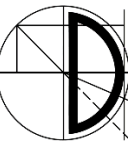
```
while ( Bedingung ) {  
    // Anweisungen  
    while ( Bedingung ) {  
        if (...) {  
            break outerLoop;  
        }  
    }  
    // Anweisungen innerhalb der äußeren Schleife  
}  
// 1. Anweisung nach äußerer Schleife
```

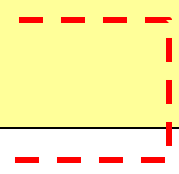
- Springt an das Ende der aktuellen Schleifeniteration
 - Kann innerhalb von **`for-`**, **`while-`**, oder **`do-while-`** Anweisungen auftreten
 - Bei **`while-`** und **`do-while-`** Anweisungen wird danach die Bedingung überprüft und entschieden ob eine weitere Iteration der Schleife ausgeführt wird
 - Bei **`for-`** Anweisungen wird die Update-Liste ausgeführt und danach die Bedingung geprüft
 - Um an das Ende einer äußeren Schleife zu springen, kann in Java eine Marke (**`label`**) verwendet werden



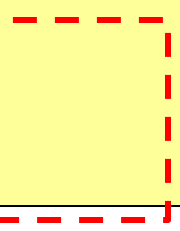
```
for (int i = 1; i < 10; i++) {  
    if (i == 3) {  
        continue;   
    }  
    IO.println(i) ;  
}  
  
int j = 0;  
while (j < 10) {  
    j++;   
    if (j == 6) continue;  
    IO.println(j) ;  
}
```



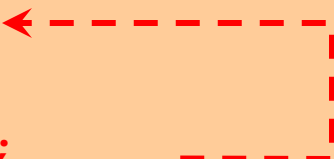
```
while (...) {  
    ...  
    if (...)  
        break;  
}
```



```
for (i = 0; i < 100; i++) {  
    if (...)  
        break;  
    ...  
}
```



```
while (...) {  
    if (...)  
        continue;  
    ...  
}
```

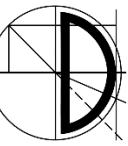


```
for (i = 0; i < 10; i++) {  
    if (...)  
        continue;  
    ...  
}
```



- Die Anweisung **break** beendet die Ausführung der innersten (das **break** umschließenden) Schleife
- Die Anweisung **continue** beendet den aktuellen Durchlauf der innersten (das **continue** umfassenden) Schleife

Reminder



Die Vorlesung nächste Woche am 07.11.2025 findet im Turing-Hörsaal (Informatikgebäude M2) statt!

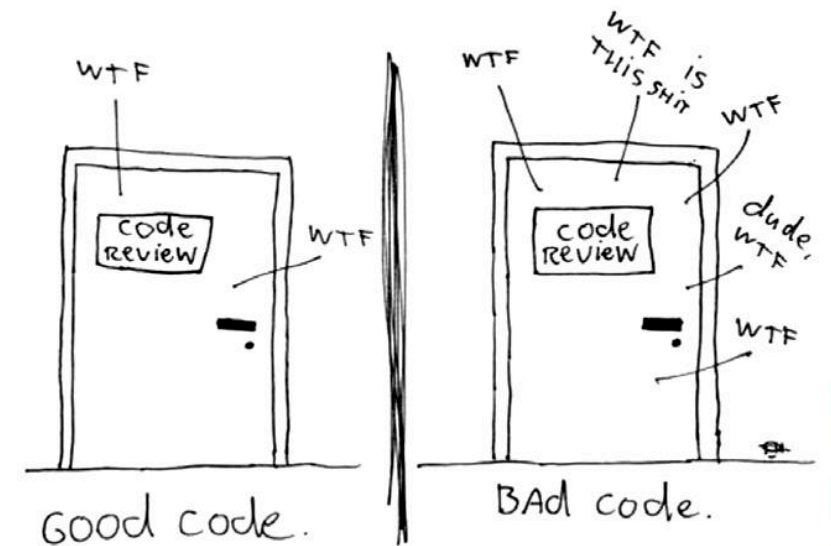
Parallel laufen die Vorbereitungen für den Science-Slam



Schon gewusst, dass Compiler manchmal Schleifen „abrollen“?

Da bei Schleifen „teure“ Bedingungen geprüft werden müssen, kann es effizienter sein, den Code des Schleifenkörpers einfach zu duplizieren. Diese Technik wird von Compilern ggf. automatisch angewendet, sodass der übersetzte Quellcode nicht genau dem vom Programmierer geschriebenen entsprechen muss. Es ist nur eine von vielen sogenannten *compiler optimizations*.

The ONLY valid measurement
of code quality: WTFs/minute



The image features a background of a classical building facade with columns and arches, likely a part of the Julius-Maximilians-Universität Würzburg. On the left, there is a blue-tinted area with a white logo consisting of a stylized 'J' and 'M' forming a square. The text 'Julius-Maximilians-' is in a smaller font above 'UNIVERSITÄT WÜRZBURG' which is in a large, bold, white sans-serif font.

Julius-Maximilians-
**UNIVERSITÄT
WÜRZBURG**

Fragen?

Vielen Dank für Ihre Aufmerksamkeit